

Full length article

Privacy-preserving path constrained shortest distance queries on encrypted graphs[☆]Yuwei Duan^a, Jingjuan Yu^b, Minghui Wang^b, Shundong Li^{b,*}^a School of Artificial Intelligence, Jiyuan Vocational and Technical College, Jiyuan, Henan 459000, China^b School of Artificial Intelligence and Computer Science, Shaanxi Normal University, Xi'an 710119, China

ARTICLE INFO

Keywords:

Graph data
Blockchain
Path constrained shortest distance query
Privacy preserving
Searchable encryption

ABSTRACT

The path-constrained shortest-distance (PCSD) query on a graph, which computes the shortest distance between two nodes while satisfying constraints on intermediate nodes, is widely used for optimized route planning in real-time personalized navigation services, where query efficiency is critical. Owners of large-scale graphs commonly outsource their data in the form of encrypted graphs to cloud servers, leveraging reliable computing resources while preserving the privacy. Nevertheless, shortest-distance queries with node constraints on encrypted graphs have not been explored. Repeated queries in the existing schemes, which support only privacy-preserving 2-hop shortest-distance queries, can produce incorrect results and leak private information. Furthermore, in the current encrypted-graph outsourcing schemes, graph owners must share their private keys, compromising key confidentiality. To overcome these challenges, we formalize the PCSD query and propose a secure, efficient, and accurate scheme called the PCSD scheme, which supports PCSD queries on encrypted graphs without revealing the graph topology. We then establish a graph-outsourcing query system that preserves the private key of the graph owner. PCSD provides verifiable query results using blockchain technology and its robustness is demonstrated in a security analysis. During experiments on encrypted graphs, the average query efficiency was up to 1.9×10^4 times higher in PCSD than in the existing whole-graph shortest-distance query schemes.

1. Introduction

Intelligent transportation systems and location-based services have rapidly advanced in recent times, necessitating shortest-distance queries in applications such as navigation (Tatit et al., 2024), logistics optimization (Wei, 2022), and traffic management (Gong et al., 2024). Because users' routes in practical scenarios must frequently pass through designated intermediate nodes (e.g., charging stations or delivery hubs), path-constrained shortest-distance queries (PCSDQs) have attracted wide interest. The PCSDQ problem involves computing the shortest distance between two nodes under predefined path constraints.

PCSDQs are essential for improving route planning efficiency, reducing the operational costs, and enhancing the service quality of modern transportation and emergency response systems. However, the expanding scale and complexity of road networks has substantially increased the computational burdens of storing and processing graph data. The burden on graph owners (GOs) can be alleviated by cloud-based outsourcing, which is widely adopted to improve the efficiency and scalability of querying (Kumari et al., 2017; Hu et al., 2023;

Selvaraj et al., 2023; Gad-Elrab and Noaman, 2020; Li, 2020). However, outsourcing graph data and routing queries to cloud servers (CSs) raises privacy concerns because the graph topology, edge weights, and user trajectories may contain sensitive information. Consequently, privacy-preserving shortest-distance queries over encrypted graphs have become a critical research challenge, particularly in the context of PCSDQ.

The existing schemes for encrypted shortest-distance queries are primarily designed for threshold-constrained queries (Zhang et al., 2020; Wang et al., 2022; Shen et al., 2017, 2019) or 2-hop shortest-path queries, a basic case with limited applicability. Research targeting PCSDQs remains scarce. Meng et al. (2015) proposed an approximate scheme for threshold-constrained shortest distance queries, and Wang et al. (2017) used Paillier homomorphic cryptosystems, PRFs, and garbled circuits to perform shortest distance queries and used a server to store the queried shortest distances to support dynamic updating of graphs. Liu et al. (2018) combined order-preserving encryption with 2-hop cover label (2HCL) indexes. Luo et al. (2022) and Ge et al. (2021)

[☆] This work is supported by National Key Research and Development Program of China, No. 2022YFB2703001.

* Corresponding author.

E-mail address: shundong@snnu.edu.cn (S. Li).

Table 1
Scheme comparison.

Reference	Query hop	Accurate result	Protect graph topology	Support other users query
GRECS (Meng et al., 2015)	2	×	×	×
SecGDB (Wang et al., 2017)	2	✓	×	×
Sun et al. (2023)	2	✓	✓	×
Luo et al. (2022)	$n - 1$	×	✓	×
SPPS (Ge et al., 2021)	$n - 1$	×	✓	✓
GESDQ (Guo and Sun, 2024)	$n - 1$	✓	✓	×
Our scheme	$n - 1$	✓	✓	✓

enhanced the accuracy and privacy protection of encrypted shortest-distance queries but produced only approximate results. To date, only the Graph Encryption for Shortest-distance Queries (GESDQ) scheme of Guo and Sun (2024) supports accurate whole-graph shortest-distance queries, but inherent design limitations prevent the direct extension of GESDQ to efficient PCSDQ support. Table 1 compares our scheme with the existing works in detail.

The major limitations of the existing schemes for shortest-distance queries on encrypted graphs are outlined below.

First, repeated queries cannot guarantee accuracy. Most of the whole-graph privacy-preserving schemes produce approximate results to balance the efficiency–accuracy tradeoff. Consequently, repeated queries accumulate errors, and the correctness of 2-hop schemes is not guaranteed for paths longer than 2 hops.

Second, repeated queries compromise the privacy of the existing schemes. Multiple distance acquisitions can reveal the shortest paths and expose the graph topology, violating the encryption requirements. Many existing approaches rely on symmetric searchable encryption, requiring the GO to share a private key with querying users for decryption. This action exposes the GO's private key, which is not allowed in reality.

Third, the existing schemes do not efficiently compute the shortest-distance queries demanded in real-time applications such as navigation. GESDQ enumerates all paths and computes distances using a distributed double-trapdoor encryption system. For a graph with n nodes and k path nodes, the complexity of this computation is $O(k \cdot n^3)$. Such rapid growth with graph size severely limits the scalability of the GESDQ method. Efficient, accurate, and privacy-preserving path-constrained shortest-distance queries are therefore urgently needed.

To address these challenges, we propose PCSD, a privacy-preserving PCSDQ scheme that leverages 2-hop covering labels (2HCL) to construct a shortest-distance matrix M . Besides being more efficient than iterative adjacency matrix computations, PCSD supports accurate path-constrained queries while revealing only the graph size. To protect key privacy, we design a generic outsourcing system enabling querying without accessing the private key of the GO. To our knowledge, PCSD is the first scheme supporting privacy-preserving path-constrained shortest-distance queries. Our contributions are summarized below.

- (1) **Accurate path-constrained queries:** We formalize PCSDQs on encrypted graphs and propose a scheme (PCSD) that supports queries through k ($k \in [0, n]$) nodes. By iteratively constructing M with 2HCL, PCSD ensures accurate shortest distances, avoiding the inaccuracy inherent in existing schemes.
- (2) **Privacy preservation:** We encrypt M using the Okamoto–Uchiyama (OU) cryptosystem with additive homomorphism, ensuring confidentiality of the graph data and topology. QUs can access only encrypted results, and the GO's private key is fully protected, enabling secure multiuser queries.

- (3) **High efficiency:** PCSD accelerates whole-graph shortest-distance computation via 2HCL and optimizes queries using M , enabling efficient privacy-preserving shortest-distance queries in cloud environments.
- (4) **Security and performance validation:** We demonstrate the security of PCSD on standard simulation paradigms. Theoretical and experimental results show that the query time depends only on the number of path nodes, not on the total graph size. Compared to existing schemes, our scheme improves the query efficiency by at least 1.9×10^4 times.

The remainder of this paper is structured as follows: Section 2 reviews the related work and Section 3 provides the preliminaries and problem definitions. Section 4 details the construction process of the PCSD scheme. Sections 5 and 6 analyze the security and efficiency of PCSD, respectively. The paper concludes with Section 7.

2. Related work

Computing the shortest distance between two nodes in a graph is a fundamental problem in graph theory, and numerous effective solutions have been proposed in the literature (Wei, 2010; Qiao et al., 2012; Sommer, 2014; Zhu et al., 2013). These studies primarily focus on optimizing graph traversal algorithms to enable efficient computation over large-scale graphs. However, such optimization techniques are designed for plaintext settings and cannot be directly transformed into ciphertext operations. Consequently, they are not directly applicable to shortest distance queries over encrypted graphs.

The rapid development of outsourcing computation has created new opportunities for data sharing, encouraging data owners to monetize their datasets. However, as such data often contain sensitive and competitively valuable information, privacy protection has become a critical concern. To support query services while preserving confidentiality, data owners typically upload their graphs to CSs in an encrypted form. As encryption inevitably restricts the types of performable operations on the data, efficient and secure query mechanisms over encrypted graphs are urgently demanded.

Graph encryption was first conceptualized by Chase and Kamara (2010), who proposed encrypting the graph topology to allow privacy-preserving computation before outsourcing it to a server. Subsequently, Meng et al. (2015) investigated shortest-distance queries over encrypted graphs and proposed three schemes based on symmetric and homomorphic encryption. However, these schemes compute only the approximate distance, not the exact shortest distances. To resolve these limitations (Meng et al., 2015), Liu et al. (2018) proposed the GENOA scheme based on the 2HCL index. Their approach determines the relationship between two sums under the following rule: if $a \leq c$ and $b \leq d$, then $a + b \leq c + d$; if $a \geq c$ and $b \geq d$, then $a + b \geq c + d$. However, this rule fails when $a > c$ but $b < d$, or $a < c$ but $b > d$. Although GENOA is superior to GRECS (Meng et al., 2015), its accuracy is suboptimal.

To further improve the accuracy, Guo and Sun (2024) employed a distributed two-trapdoor public-key cryptosystem that supports ciphertext addition and comparison, providing accurate query results under strong security guarantees. However, this cryptosystem requires a more complex key management than conventional public-key systems, and the sophisticated comparison and computation procedures incur a large computational overhead. Moreover, this scheme allows only querying by data owners, because decryption of query results requires the GO's secret key. Therefore, the practicality of this scheme is severely limited.

To reduce the computational overhead, Sun et al. (2023) utilized the 2HCL index and adjacency linked lists for shortest-distance computations in large-scale road networks. Their scheme encrypts nodes and edges using a pseudo-random function (PRF) and symmetric encryption. However, it leaks node-degree information. Furthermore, because it relies on symmetric encryption, only the data owner can issue queries, which remains a substantial limitation.

Zhang et al. (2020) proposed a privacy-preserving shortest-distance query scheme with a single weight constraint based on 2HCL. Their scheme adopts a PRF for node encryption and threshold Paillier encryption with additive homomorphism to protect distance information. Leveraging cooperation between a storage server and a computing server, their scheme achieves unlinkable query trapdoors, thereby enhancing both the security and result accuracy. Building upon Privacy-preserving Graph encryption for Accurate constrained Shortest distance (PGAS), Wang et al. (2022) introduced Strong Privacy-preserving Constrained Shortest-distance (SPCS) with double constraints. SPCS stores the distance and cost information in matrix form, protecting the node-degree information and providing stronger security than PGAS. However, both PGAS and SPCS rely on integer-comparison protocols and secure minimum protocols over ciphertexts, introducing additional interaction and computational complexity. These protocols degrade the query efficiency and lengthen the response time.

With the exception of GESDQ (Guo and Sun, 2024), the aforementioned schemes are limited to shortest-distance queries with 2-hop covers. Consequently, they cannot enforce constraints under which the shortest path must pass through specific nodes. If the existing schemes are repeatedly invoked for PCSDQs, their accuracy and security may be compromised. Multiple invocations may reveal pairwise shortest distances, increasing the risk of an adversary inferring the graph's topological structure. Such information should remain confidential within a secure model of encrypted-graph queries. Therefore, achieving efficient and secure PCSDQs over encrypted graphs remains an open and challenging problem.

3. Preliminaries and problem formulation

3.1. Preliminaries

(1) **Blockchain:** BC is a distributed ledger technology that organizes data into blocks and links these blocks into a chain. Each block is validated and added by network participants known as miners, who use cryptographic methods to create a secure, transparent, and immutable system. In BC terminology, transparency means that all transaction records and data are publicly accessible and that anyone can verify their transaction histories on the BC. Immutability, meaning that a transaction recorded on the BC cannot be modified or deleted, is ensured through cryptographic techniques and consensus mechanisms. Immutability maintains the integrity and trustworthiness of the data.

Smart contracts are self-executing contracts in which the terms of the agreement are directly written into code. These contracts automatically execute, verify, or enforce the terms when predefined conditions are met. BC provides a decentralized and tamper-resistant platform for smart contracts, ensuring the security and transparency of contract execution. All actions and results of smart contracts can be verified and audited on the BC.

(2) **Path Constrained Shortest Distance Query:** Let $G = (V, E)$ be a graph on which each edge $e \in E$ with weight d represents a distance between two nodes. Along the sequence of edges $P = (e_{i1}, e_{i2}, \dots, e_{iq})$ in G , where $e_{ik} = (v_i, v_j)$, each v_i is the end node of e_{ik-1} , and each v_j is the start node of e_{ik+1} . Such a sequence is called a path in graph G .

The distance of a path $P = (e_{i1}, e_{i2}, \dots, e_{iq})$ is defined as $d_P = \sum_{j=1}^q d_{ij}$. Under this constraint, the user must specify not only the start and end nodes, but also the intermediate nodes v_k , to determine the shortest-path distance.

As shown in Fig. 1, G is a weighted graph where the edge weights represent the distances between two nodes. With no path-node constraints, the shortest distance between nodes 1 and 4 is 4 and the shortest path is $(1 \rightarrow 0 \rightarrow 2 \rightarrow 4)$ (dotted line in Fig. 1(a)). If path node 3 is specified, the shortest distance between nodes 1 and 4 becomes 6, and the shortest path is $(1 \rightarrow 0 \rightarrow 3 \rightarrow 0 \rightarrow 2 \rightarrow 4)$ (dotted line in Fig. 1(b)).

(3) **2-Hop Cover Label Index:** The 2HCL index (Cohen et al., 2003), a data structure commonly used in shortest-distance queries, is fundamental for establishing distance relationships between nodes. Let

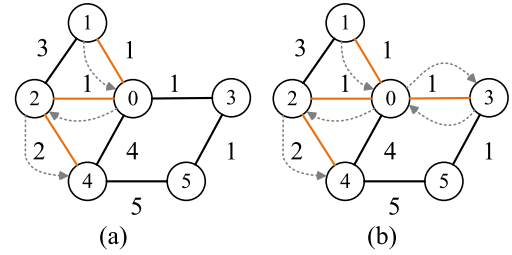


Fig. 1. Example of Path Constrained Shortest Distance Query. (a) No path constraint. (b) With path constraints.

Table 2

2HCL $L(v_i)$ of nodes in Graph G .

Node i	Label
$L(0)$	$[(0, 0), (1, 1), (2, 1), (3, 1), (4, 3), (5, 2)]$
$L(1)$	$[(0, 1), (1, 0), (2, 2), (3, 2), (4, 5)]$
$L(2)$	$[(0, 1), (1, 2), (2, 0), (3, 2), (4, 2), (5, 7)]$
$L(3)$	$[(0, 1), (1, 2), (2, 2), (3, 0), (4, 5), (5, 1)]$
$L(4)$	$[(0, 3), (1, 5), (2, 2), (3, 5), (4, 0), (5, 5)]$
$L(5)$	$[(0, 2), (2, 7), (3, 1), (4, 5), (5, 0)]$

$G = (V, E)$ be a graph composed of a node set V and an edge set E . Each node $v_i \in V$ is associated with a label $L(v_i)$, which consists of multiple pairs $(v_j, d_{i,j})$. Here, $(v_j, d_{i,j})$ denotes that the shortest distance from v_i to v_j within two hops is $d_{i,j}$. The labels $L(v_i)$ of the nodes on graph G in Fig. 1(a) are presented in Table 2. To compute the shortest distance between two nodes, we can consider the common nodes in their respective labels:

$$d_{u,v} = \min_{k \in L(u) \cap L(v)} \{d_{u,v}, d_{u,k} + d_{k,v}\} \quad (1)$$

The complete shortest-distance relationships between the nodes in graph G cannot be constructed solely using the 2-hop index. When $|V| = n$, after $\lceil \log_2 n \rceil$ iterations, all reachable nodes can be connected.

During the first iteration, we derive the 4-hop shortest-distance index from the 2-hop index. Specifically, if node u can reach node v within 2 hops, and node v can reach node w within 2 hops, then node u can reach node w within 4 hops. Expanding this process, we can obtain 8-hop, 16-hop, and other shortest-distance indices until the indexing covers the maximum diameter of the graph. That is, up to $n - 1$ hops are needed to connect any two nodes.

The x -hop shortest-distance index stores the shortest distance between any two nodes within at most x hops, ensuring that the shortest distance between any pair of nodes is accurately determined. As each iteration doubles the hop count, at most $\lceil \log_2 n \rceil$ iterations will cover the graph's maximum diameter.

(4) **Okamoto-Uchiyama Cryptosystem:** Widely employed in privacy-preserving computations, the OU algorithm is an asymmetric encryption scheme based on the factorization assumption. The OU cryptosystem consists of three primary algorithms: key generation, encryption, and decryption.

Key Generation: Given a security parameter ξ , the algorithm randomly generates two large prime numbers p and q , each with ξ bits. It then computes $n = p^2 q$ and selects a generator g of Z_n^* such that $g_p = g^{p-1} \mod p^2$ has order p . Let $h = g^n \mod n$. The public and private keys are denoted as $pk = (n, g, h)$, and $sk = (p, q)$, respectively.

Encryption: Given a message $0 < m < 2^{\xi-1}$ and public key pk , encryption selects a random number $r \in Z_n$ and computes the ciphertext C :

$$C = g^m h^r \mod n. \quad (2)$$

Decryption: Given a ciphertext C , decryption computes $C_p = C^{p-1} \mod p^2$, $g_p = g^{p-1} \mod p^2$, and plaintext

$$m = \frac{L(g_p)}{L(C_p)} \mod p \quad (3)$$

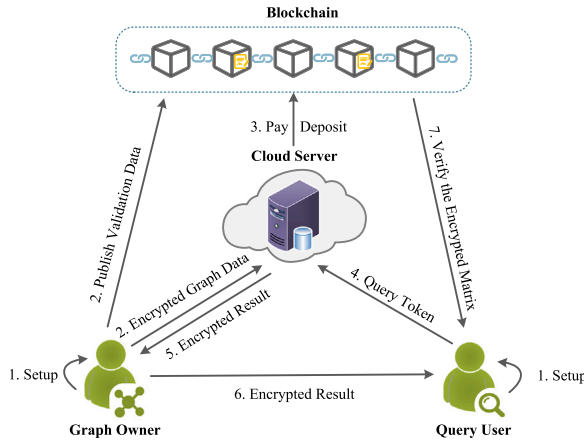


Fig. 2. System model of PCSD scheme.

where $L(x) = \frac{x-1}{p}$.

Additive homomorphism: OU encryption algorithm is additively homomorphic. For plaintext m_1, m_2 ,

$$E(m_1) = g^{m_1} h^{r_1} \bmod n,$$

$$E(m_2) = g^{m_2} h^{r_2} \bmod n.$$

We then have:

$$\begin{aligned} E(m_1) \cdot E(m_2) &= g^{m_1} h^{r_1} \cdot g^{m_2} h^{r_2} \bmod n \\ &= g^{m_1+m_2} h^{r_1+r_2} \bmod n \\ &= E(m_1 + m_2). \end{aligned} \quad (4)$$

3.2. Problem formulation

(1) **System Model:** Fig. 2 shows the system model of the proposed PCSD scheme. The main entities are described below:

Cloud Server (CS): Equipped with ample storage space and computing power, the CS handles the graph-data storage and responds to query requests.

Graph Owner (GO): The graph owner possesses the original data of graph G and outsources it to the CS for storage and computation.

Query User (QU): The QU queries the outsourced graph G and performs calculations on the CS.

Blockchain (BC): The BC platform deploys smart contracts, stores the GO metadata, and supports QU verifications of CS.

(2) **Security model:** GO and QU are assumed as semi-honest entities who will honestly execute the protocol. The GO will honestly process the graph and send ciphertext data to the CS according to the protocol. The QU will honestly generate query tokens to the CS. However, both GO and QU are interested in the information of the other; that is, GO desires to know the query nodes (including path nodes) of the QU while the QU is interested in the graph topology of the GO. As semi-honest entities, CSs try to infer the plaintext of encrypted datasets and query nodes from the information uploaded by the GO and QU. CS will not collude with either GO or QU to obtain the private information of the other party.

(3) Security Definition:

Semi-honest model: A semi-honest model assumes that all participants in the system act semi-honestly. Semi-honest parties adhere to the prescribed protocol steps during execution but may attempt to obtain additional information about other participants. In this paper, the CS, GO, and QU are assumed semi-honest and the CS colludes with neither the GO nor the QU. Specifically, the CS strictly executes the protocol when accessing and processing the encrypted matrix but may attempt to infer private information (e.g., graph topology, query nodes) from the observed execution traces, including matrix access patterns.

Table 3

List of notations.

Notation	Meaning
$G(V, E)$	Original graph data
n	Number of nodes of G
m	Number of edges of G
M	The shortest distance matrix
L	The adjacent linked list
pk_1, sk_1	The public/private keys of the graph owner
pk_2, sk_2	The public/private keys of the query user
H, K	Pseudo random function and corresponding key
$\llbracket x \rrbracket$	The ciphertext corresponding to the plaintext x

Security in the semi-honest model: The security of a protocol under the semi-honest model is typically formalized using the simulation-based paradigm (Goldreich, 2004): A protocol π is considered secure if whatever a party can obtain from the protocol execution can also be simulated by a polynomial-time simulator using only the party's explicitly declared leakage.

The view of party i during the execution of inputs $(x_1, \dots, x_n)$ is denoted as

$$view_i^\pi(x_1, \dots, x_n) = (x_i, r_i, M_i^1, \dots, M_i^t),$$

where r_i is the randomness generated by party i during the protocol execution, and M_i^j ($j = 1, 2, \dots, t$) is the j th message received by party i .

Definition 1 (Leakage Function). For each party i , the leakage function $\mathcal{L}_i$ specifies the information that party i is allowed to learn during the execution of the protocol.

Formally,

$$\mathcal{L}_i : (x_1, x_2, \dots, x_n) \rightarrow \mathcal{Y}_i$$

maps the global inputs $(x_1, \dots, x_n)$ to the information revealed to party i .

The value $\mathcal{L}_i(x_1, \dots, x_n)$ captures all intentionally revealed information, such as protocol execution results, access patterns, etc.

Definition 2 (Security with Leakage). Let $\mathcal{L}_i(x_1, \dots, x_n)$ denote the formally declared leakage function for party i . The protocol π is said to securely compute function f if, for each party i , there exists a probabilistic polynomial-time simulator S_i such that

$$\{S_i(\mathcal{L}_i(x_1, \dots, x_n))\}_{x_1, \dots, x_n} \stackrel{c}{\equiv} \{view_i^\pi(x_1, \dots, x_n)\}_{x_1, \dots, x_n},$$

where $\stackrel{c}{\equiv}$ denotes computational indistinguishability. This definition ensures that party i learns only the input, output, and explicitly allowed leakage from the protocol execution.

4. Path constrained shortest distance query scheme

The scheme consists of six stages: (1) Setup; (2) Graph preprocessing and encryption; (3) Verification Mechanism Establishment; (4) Query token generation; (5) Path-constrained shortest distance query; and (6) Decryption and verification. The main symbols used in this paper are summarized in Table 3.

4.1. Overview

In PCSD, the GO executes the shortest-distance matrix-generation algorithm (Algorithm 1), which retrieves the shortest-distance matrix M and implements the secure graph encryption algorithm (Algorithm 2). When the QU wants to query the path distance of specified path nodes, it generates a query token for the query nodes to initiate a PCSDQ. The CS retrieves the computed result in ciphertext and sends it to the GO. Finally, assisted by the GO, the QU decrypts the ciphertext to obtain the query result.

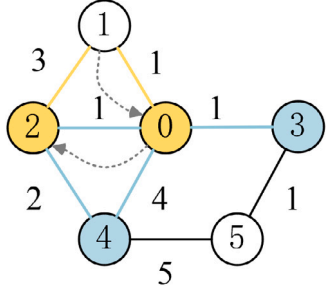


Fig. 3. Example of Index Iteration.

4.2. Setup

In this step, the GO inputs the security parameter ξ and executes the key-generation algorithm of the OU encryption algorithm, thereby obtaining public and private key pairs (pk_1, sk_1) . Next, the GO generates a PRF key K for generating the pseudo-identity of the nodes and sends K to QUs, enabling them to generate the pseudo-identities of query nodes.

4.3. Graph preprocessing and encryption

The GO preprocess the original graphs to facilitate queries. We first introduce procedures for generating the shortest-distance matrix M . We then introduce Algorithm 2 (the Secure Graph Encryption Algorithm), which protects the topology of graph $G = (V, E)$.

(1) **Generation of the shortest-distance matrix:** The original graph data are first converted to an adjacency list, which stores the edges in the graph. Subsequently, the shortest distance between two nodes is continuously updated by extending the reachable scope between nodes. This process continues until the shortest distance between any two nodes on the whole graph is obtained. The distance is updated as $d_{u,v} = \min(d_{u,v}, d_{u,k} + d_{k,v})$, where k is an element in the intersection of the sets of neighbor nodes of nodes u and v . In other words, the shortest distance between node u and other nodes is updated through neighbor node k .

For example, in the graph in Fig. 3, the neighbors of node 1 are nodes 0 and 2. During the first iteration, the shortest distance between node 1 and other reachable nodes is updated by considering the neighbors of nodes 0 (nodes 1, 2, 3, and 4) and 2 (nodes 0 and 4). Calculating $d_{1,2} = \min(3, 1 + 1) = 2$, we find that the shortest distance between nodes 1 and 2 is adjusted through node 0 (dashed path in Fig. 3).

Iterating the outward diffusion of neighbor nodes, a path between the two farthest-separated reachable nodes in an n -node graph G is found after $\lceil \log_2 n \rceil$ iterations at most. Paths can be established between all reachable nodes. If two nodes remain unreachable within the shortest-distance index covered by $n - 1$ hops, then no shortest distance exists between those nodes.

To protect the degree of nodes in a graph, the index L needs to be converted to the shortest distance matrix M : if v_k exists within the index node list of v_i , then $M[i][k] = d_{i,k}$; otherwise, $M[i][k] = \infty$ (we denote ∞ as 2^{30}). The shortest distance matrix generation algorithm is given as Algorithm 1.

(2) **Secure graph encryption:** The GO encrypts each element in the shortest-distance matrix $M_{n \times n}$ using pk_1 , obtaining an encrypted matrix $\llbracket M \rrbracket$ that protects the privacy of the shortest-distance result. To prevent leakage of the QU's query nodes, the GO also computes the pseudo-random identity $H(v_i, K)$ for each node $v_i \in V$ using the PRF. Finally, the GO outsources the encrypted matrix $\llbracket M \rrbracket$ along with a pseudo-identity list of the nodes to the CS.

Algorithm 1 Graph Index Iteration.

Input: Original graph data $G(V, E)$

Output: shortest distance matrix M .

```

1: Initialize  $L \leftarrow 2 - \text{hop}(G)$ ,  $M \leftarrow$  empty matrix
2:  $n = |V|$ 
3: for  $i = 1$  to  $\lceil \log_2 n \rceil$  do
4:   for  $v \in V$  do
5:     for  $v$ 's neighbor node  $k$ , through  $k$ 's neighbor node  $u$  to update  $d_{v,u}$ :
        $d_{v,u}^i = \min(d_{v,u}^{i-1}, d_{v,k} + d_{k,u})$ 
6:   end for
7: end for
8: Convert index  $L$  to matrix  $M$ : if  $u \in L(v)$ , then  $M[v][u] = d_{v,u}$ ,
   otherwise  $M[v][u] = \infty$ 
9: return Shortest distance matrix  $M$ 

```

Algorithm 2 GraphEnc

Input: Shortest distance matrix M , public key pk_1 , PRF and corresponding key H, K

Output: Ciphertext matrix $\llbracket M \rrbracket$, node pseudo-identity list V_G

```

1: for  $i = 1$  to  $n$  do
2:   for  $j = 1$  to  $n$  do
3:     GO use  $pk_1$  to encrypt elements  $a_{ij}$  in the shortest distance
       matrix  $\llbracket M \rrbracket$  to obtain ciphertext  $\llbracket a_{ij} \rrbracket$ 
4:   end for
5: end for
6: for  $i = 1$  to  $n$  do
7:   GO computes  $\llbracket V_i \rrbracket = H(v_i, K)$ 
8:    $V_G$  append  $\llbracket V_i \rrbracket$ 
9: end for
10: return Encrypted shortest distance matrix  $\llbracket M \rrbracket$ , pseudo-identity
    list  $V_G$ 

```

4.4. Establishment of the verification mechanism

To ensure the correctness of the outsourced stored data, the GO uploads the shortest distance matrix M to the CS while simultaneously uploading the verification data $\Phi_v = \text{hash}(\llbracket M \rrbracket)$ to the BC.

4.5. Query token generation

When performing a query, a QU must generate a query token for its list of query nodes. The QU first inputs a security parameter, ξ , and runs the key-generation algorithm of the OU encryption scheme, which returns a pair of public and private keys (pk_2, sk_2) . Using the public key pk_2 and the key K sent by the GO, the QU generates pseudo-identities for the start node v_i , the end node v_j , and the intermediate path nodes $v_{k1}, v_{k2}, \dots, v_{kl}$.

To prevent the GO from deducing the query nodes from the query results, the QU randomizes the query result by selecting a random number $R \in [0, n]$ and computing $\llbracket R \rrbracket = g^R h^r \bmod n$. The public key pk_2 , list of node pseudo-identities, and randomized result $\llbracket R \rrbracket$ are simultaneously uploaded to the CS for initiating the query. The query-token generation algorithm is given as Algorithm 3.

4.6. Path-constrained shortest-distance query

After receiving a query token $Q = (V_q, \llbracket R \rrbracket, pk_2)$ from the QU, the CS performs a path-constrained query on the encrypted graph. The query is processed as follows:

Algorithm 3 TokenGen

Input: Query nodes $q_v = (v_i, v_{k1}, \dots, v_{kl}, v_j)$, PRF and corresponding key H, K

Output: Query token Q

- 1: Compute the pseudo-identities of all queried nodes:
 $V_q = (H(v_i, K), H(v_{k1}, K), \dots, H(v_{kl}, K), H(v_j, K))$
- 2: Select a random number $R \in [0, n]$, compute $\llbracket R \rrbracket = g^R h^r \bmod n$
- 3: $Q = (V_q, \llbracket R \rrbracket, pk_2)$
- 4: **return** Query token Q

- (1) After comparing the query-node list sent by the QU with the node list sent by the GO, the CS retrieves the distances $d_1, d_2, \dots, d_m$ from the corresponding rows and columns of $\llbracket M \rrbracket$.
- (2) The CS multiplies the ciphertext d_i (where $i \in [1, m]$) and computes the query result $\llbracket c \rrbracket = \prod_{i=1}^m d_i \cdot \llbracket R \rrbracket$.
- (3) The CS sends the query result $\llbracket c \rrbracket$ to the GO.
- (4) Using the private key sk_1 , the GO decrypts $\llbracket c \rrbracket$ and then re-encrypts it with pk_2 , obtaining the ciphertext $\llbracket e \rrbracket$. Afterward, the GO sends $\llbracket e \rrbracket$ to QU to complete the query response.

Algorithm 4 Secure PCSD Query (Graph Query)

Input: The encrypted shortest distance matrix $\llbracket M \rrbracket$, pseudo-identity list V_G , and query token $Q = (V_q, \llbracket R \rrbracket, pk_2)$

Output: Encrypted shortest distance result $\llbracket c \rrbracket$

- 1: **Cloud Server:**
- 2: **for** $i = 1$ to m **do**
- 3: Computes the indices i and j of $H(v_i)$ and $H(v_j)$ in the pseudo-identity list V_G of nodes.
- 4: Locates and restores the distance d_i of the row index i and the column index j in M .
- 5: **end for**
- 6: Computes $\llbracket c \rrbracket = \prod_{i=1}^m d_i \cdot \llbracket R \rrbracket$ to get the result $\llbracket c \rrbracket$.
- 7: Sends $\llbracket c \rrbracket$ and pk_2 to the graph owner.
- 8: **Graph Owner:**
- 9: Decrypts the result $\llbracket c \rrbracket$ using private key sk_1 to obtain the plaintext $c = d + R$.
- 10: Encrypts $d + R$ using the query user's public key pk_2 to obtain the ciphertext $\llbracket e \rrbracket = \llbracket d + R \rrbracket$ and send it to the query user.

Algorithm 4 gives the query algorithm (Graph Query). The shortest-distance matrix M uploaded by the GO stores the shortest distance covered by $n - 1$ hops between any two nodes in the graph. To obtain the shortest distance between nodes v_i and v_j , the algorithm should compute the indices i and j of $H(v_i)$ and $H(v_j)$ in the pseudo-identity list V_G of nodes. The ciphertext representing the shortest distance between v_i and v_j is denoted as $\llbracket M \rrbracket[i][j]$. Under the above rules (lines 2–5 of Algorithm 4), the CS retrieves m ciphertext distances d_i (where $i \in [1, m]$) for the nodes in V_q . The CS multiplies the ciphertexts d_i and then randomizes them by multiplying the multiplied d_i with the $\llbracket R \rrbracket$ uploaded by the QU. The CS sends the result e and the key pk_2 to the GO.

The GO decrypts $\llbracket e \rrbracket$ to obtain the plaintext $(d + R)$. Lacking any information on R , the GO cannot infer d so the query results remain private. Subsequently, the GO re-encrypts $d + R$ using the QU's pk_2 and sends it to the QU to complete the query.

4.7. Decryption and verification

Decryption: After receiving the result from the GO, the QU decrypts the ciphertext $\llbracket e \rrbracket$ using sk_2 , thus obtaining the query result e . Finally, the QU obtains the real distance $d = e - R$. If $d > 2^{30}$, the query path

is unreachable, indicating that no shortest distance exists; otherwise, d represents the shortest distance of the query path.

Verify the encrypted matrix: When a QU initiates the verification, the CS sends the matrix $\llbracket M \rrbracket$ used in the computation to the QU for validation. The QU computes $\Theta_v = \text{hash}(\llbracket M \rrbracket)$ and compares Θ_v with Φ_v on the BC. A match indicates that the matrix has not been modified; a mismatch indicates that the CS has behaved dishonestly and its deposit will be deducted by the smart contract.

Although lightweight cryptographic signatures and

Merkle Trees can provide efficient integrity verification with relatively low overhead, their security guarantees typically rely on the presence of a trusted entity to publish commitments or resolve disputes. In decentralized outsourcing environments where no fully trusted third party is assumed, such approaches do not inherently provide decentralized supervision or enforceable economic accountability.

By contrast, the BC-based verification mechanism in our scheme serves as a decentralized trust infrastructure rather than merely an integrity verification tool. The hash digest of $\llbracket M \rrbracket$ is recorded on-chain, ensuring immutable and publicly auditable commitment. In the event of inconsistency, a predefined smart contract can automatically execute penalty rules, such as deducting the deposit of a dishonest CS. Although BC requires more overhead than lightweight alternatives, the additional overhead is maintained moderate by storing only compact hash digests on-chain and by limiting interactions with the BC only to matrix updates or verification disputes. Meanwhile, BC enhances the auditability and accountability of the scheme.

5. Analysis of the scheme**5.1. Correctness analysis**

A lemma in graph theory (Bondy and Murty, 2008) states that if $P(i)$ denotes the shortest path from v_1 to v_i in graph G and v_j is a node on $P(i)$, then the subpath $P(j)$ from v_1 to v_j is also the shortest path.

By the additive property of shortest paths, we conclude that on a given path $P = v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4$, each segment $v_1 \rightarrow v_2$, $v_2 \rightarrow v_3$, and $v_3 \rightarrow v_4$ corresponds to a shortest path between successive nodes. Therefore, the total shortest distance of the entire path is exactly the sum of the shortest distances of the individual line segments, independently of the other paths. That is, the total shortest distance along path $P = v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4$ can be represented as the sum of the shortest distances along each segment:

$$d(1, 4) = d(1, 2) + d(2, 3) + d(3, 4).$$

The matrix entry $d(i, j)$ already represents the shortest path distance between the node pair (v_i, v_j) , considering all possible intermediate nodes. Therefore, $d(i, j)$ is globally optimized and represents the shortest distance.

As the shortest-path matrix has been pre-computed and each element $d(i, j)$ represents the optimized shortest distance, the cumulative shortest distance along path P is correctly given by the sum of its segment distances.

5.2. Security analysis

We analyze the security of the PCSD scheme under the leakage framework. Recall that a leakage function specifies the information that each party is allowed to learn during protocol execution (Definition 1). Based on this definition, we specify the leakage functions of the involved parties as follows.

The leakage function of the cloud server (CS) is

$$\mathcal{L}_{CS} = (n, t, AP),$$

where n denotes the size of the encrypted distance matrix $\llbracket M \rrbracket$, t denotes the number of pre-defined query nodes, and $AP = (i, j)$

represents the matrix access pattern observed during query execution, i.e., the sequence of row and column indices accessed by the CS in $\llbracket M \rrbracket$.

The leakage function of the graph owner (GO) is

$$\mathcal{L}_{GO} = (x_{GO}, f_{GO}),$$

where $x_{GO} = \llbracket M \rrbracket$ and $f_{GO} = \emptyset$.

The leakage function of the query user (QU) is

$$\mathcal{L}_{QU} = (x_{QU}, f_{QU}),$$

where x_{QU} denotes the query input (V_q, R) and $f_{QU} = d$, where d is the shortest distance corresponding to the queried node set V_q . Under the protocol, only the QU obtains the final output, while the GO and CS produce no outputs, i.e., $f_{GO} = f_{CS} = \emptyset$.

During protocol execution, the CS receives the encrypted matrix $\llbracket M \rrbracket$ from the GO and the query token Q from the QU, retrieves the corresponding ciphertext entries, performs homomorphic multiplications on the encrypted shortest-distance values, and returns the resulting ciphertext to the GO. Since the CS does not possess the private key of the GO or the PRF key K , and the OU encryption scheme provides semantic security, the ciphertexts are computationally indistinguishable. Therefore, the CS cannot learn any information about the graph data, query nodes, or shortest distances beyond the information captured by $\mathcal{L}_{CS}$.

The GO receives the intermediate ciphertext e returned by the CS and decrypts it to obtain $d + R$. Because R is randomly generated by the QU, the GO cannot recover the true distance d . Moreover, the GO does not observe the matrix access pattern AP or the query token details.

Following the simulation-based security paradigm, we construct simulators S_{CS} , S_{GO} , and S_{QU} that take as input only the leakage specified by their respective leakage functions and generate simulated views of the protocol execution. If these simulated views are computationally indistinguishable from the information obtained during the actual protocol execution, then the private data of all parties are protected in the semi-honest model. Therefore, the security of the PCSD scheme in the semi-honest model is established through the following theorem.

Theorem 1. *The PCSD scheme is secure in the semi-honest model with respect to the leakage functions $\mathcal{L}_{CS}$, $\mathcal{L}_{GO}$, and $\mathcal{L}_{QU}$ defined above.*

Proof. We prove the theorem by constructing simulators for each party and showing that their simulated views are computationally indistinguishable from the corresponding real protocol views.

(1) First, we prove the security of QU's private data v_i, v_j, v_{k_a} ($a \in [1, l]$). GO receives the distance ciphertext $\llbracket c \rrbracket$ sent by the CS. The view of GO during protocol execution is:

$$\text{view}_{GO}^\pi(\llbracket M \rrbracket, Q) = (\llbracket M \rrbracket, \llbracket c \rrbracket),$$

where $\llbracket c \rrbracket$ is the distance ciphertext randomized by QU. We construct a simulator S_{GO} with input $\mathcal{L}_{GO}$. The simulator uses $c = d + R$, obtained from GO's decryption, and encrypts it under pk_1 to generate the ciphertext $\llbracket c' \rrbracket$. Let

$$S_{GO}(\mathcal{L}_{GO}) = (\llbracket M \rrbracket, \llbracket c' \rrbracket),$$

by semantic security of the OU cryptosystem, we get:

$$S_{GO}(\mathcal{L}_{GO}) \stackrel{c}{\equiv} \text{view}_{GO}^\pi(\llbracket M \rrbracket, Q),$$

ensuring that QU's private nodes and distances remain hidden.

(2) Next, we construct a simulator S_{QU} that proves the security of GO's private data, namely, the topology of the graph and the distance between corresponding nodes.

The view of QU is:

$$\text{view}_{QU}^\pi(Q, f_{QU}) = (Q, r, \llbracket e \rrbracket),$$

where r denotes QU's randomness. S_{QU} encrypts the received $f_{QU} = d$ as well as the random number R using QU's public key to obtain the ciphertext $\llbracket e' \rrbracket$, let

$$S_{QU}(\mathcal{L}_{QU}) = (Q, r, \llbracket e' \rrbracket).$$

Since the OU cryptosystem is semantically secure, so $\llbracket e \rrbracket$ is computationally indistinguishable from $\llbracket e' \rrbracket$. Therefore,

$$S_{QU}(\mathcal{L}_{QU}) \stackrel{c}{\equiv} \text{view}_{QU}^\pi(Q, f_{QU}),$$

which implies that GO's private graph topology and distance information are not revealed.

(3) Finally, we construct a simulator S_{CS} that shows how the view of the CS can be simulated from its declared leakage, implying that the CS learns no information beyond the specified leakage during protocol execution. The view of CS during protocol execution is:

$$\text{view}_{CS}^\pi(\perp) = (\llbracket M \rrbracket, Q, AP),$$

where Q denotes the PRF-generated query tokens.

The simulator S_{CS} inputs only the leakage

$$\mathcal{L}_{CS} = (n, t, AP),$$

and generates a random encrypted matrix $\llbracket M' \rrbracket$ of size n , along with a sequence of random tokens Q' consistent with t queries and access pattern AP . The output of the simulator is

$$S_{CS}(\mathcal{L}_{CS}) = (\llbracket M' \rrbracket, Q', AP).$$

By semantic security of the OU cryptosystem, $\llbracket M \rrbracket$ is computationally indistinguishable from a randomly generated ciphertext matrix $\llbracket M' \rrbracket$ of the same size. Moreover, by the pseudorandomness of the PRF, the real query tokens are computationally indistinguishable from uniformly random strings. Therefore, we have

$$S_{CS}(\mathcal{L}_{CS}) \stackrel{c}{\equiv} \text{view}_{CS}^\pi(\perp).$$

implying that the CS cannot infer any information about the plaintext matrix or the actual query nodes beyond the declared leakage $\mathcal{L}_{CS}$.

Thus, the simulated views are computationally indistinguishable from the real protocol views. Therefore, the PCSD scheme is secure in the semi-honest model. This completes the proof. ■

6. Efficiency analysis

In this section, we evaluate the proposed scheme PCSD through theoretical analysis and experiments. Since there is no existing research on the shortest distance query with path constraint on encrypted graphs, we will modify GESDQ (Guo and Sun, 2024), which supports accurate queries of the shortest distance between any two nodes on an encrypted graph, to obtain PCSD queries on encrypted graphs.

6.1. Theoretical analysis

We first analyze the computational complexity in different stages of PCSD, focusing on the computational complexities of the graph preprocess, data encryption, and query stages.

In PCSD, the GO uses shortest distance matrix to store the distance between any two nodes. The number of encryption operations is related to the number of nodes in the graph. We denote by T_e and T_p the time requirements of a modular exponential operation and a PRF operation, respectively. The OU cryptography involves two modular exponential operations per encryption. The communication complexity of the scheme is measured by the number of interactions between the parties in the system model.

(1) Computational complexity analysis

Graph preprocessing: The complexity of Algorithm 1 depends on the number of outer iterations and the computational overhead of each iteration. First, the outer layer is iterated with the 2HCL index to

Table 4
Efficiency analysis between PCSD and GESDQ.

	Encryption	Amortized	Query	Communication
PCSD	$n^2 T_e$	$\frac{n^2 T_e}{l} \rightarrow 0 (l \rightarrow \infty)$	$q T_p + 3 T_e$	3
GESDQ (Guo and Sun, 2024)	$n T_p + 2 m T_e$	$\frac{n T_p + 2 m T_e}{l} \rightarrow 0 (l \rightarrow \infty)$	$2(q-1)d_s(d_s + d_t)T_e$	$2(q-1)(d_s + d_t + 1)$

Table 5

Information of Data sets.

Dataset	Type	Nodes	Edges	Density
ego-Facebook	Undirected	4039	88 234	0.005
p2p-Gnutella05	Directed	8846	31 839	0.0004
wiki-Vote	Directed	7115	103 689	0.002
Twitch-PTBR	Undirected	1912	31 299	0.008

obtain 4-hop, 8-hop, ..., n-hop shortest distances. The total number of loops is $\lceil \log_2 n \rceil$, where n is the number of nodes in the graph. During each iteration, each node v in the graph is updated after traversing its neighboring nodes and the neighbors of its neighboring nodes. If graph G contains m edges, the computational overhead of each iteration is $O(m + n)$. Therefore, the overall time complexity of Algorithm 1 is $O(\log_2 n \cdot (n + m) + n^2)$. On sparse graphs with m close to n , the time complexity reduces to $O(n \log_2 n + n^2)$. On dense graphs with m close to n^2 , the time complexity is $O(n^2 \log_2 n + n^2)$.

Graph encryption: The GO encrypts each element in the matrix M and outsources it to the CS. For a graph with n nodes, encrypting the matrix M requires $2n^2$ modulo-exponentiations. If the number of queries is l , then the cost of encrypting the shortest distance matrix in one query (called the amortized cost) is $\frac{2n^2 T_e}{l}$. Note that as $l \rightarrow \infty$, the $\frac{2n^2 T_e}{l} \rightarrow 0$. Therefore, in the event of multiple queries, the computational overhead of the encryption step for each query is low.

Query: During the query response stage, the QU should generate a query token when initiating a query. The QU generates pseudo-identities for query nodes using a PRF and selects a random number R for re-randomization. The above operation performs q PRF computations and one modular exponential operation. The time cost of the GO in decrypting and re-encrypting the result after the query is $2 T_e$. Therefore, the computational cost of a complete query phase is $q T_p + 3 T_e$.

(2) Communication complexity analysis

In the PCSD scheme, a QU initiates a query by sending a query token to the CS. Then the CS obtains the result and sends it to the GO. Finally, GO sends the query result to the QU. In this process, there are 3 interactions between the GO, the QU, and the CS.

We have analyzed the computational complexity and the number of communications for GESDQ at the same stages, and the specific results are listed in Table 4.

(3) Storage complexity analysis

In the PCSD scheme, the GO locally computes the all-pairs shortest-distance matrix M . Each element of M is then encrypted using the OU cryptosystem, generating the encrypted matrix $\llbracket M \rrbracket$, which is uploaded to the CS to support the query. This process introduces a storage overhead of $O(N^2)$ on the cloud side. To quantify this overhead, we performed storage measurements (in Python) on a laptop running Windows 11 (64-bit).

Theoretically, the ciphertext size of the OU cryptosystem depends on the bit length of modulus $n = p^2 q$. When the prime p is 1024 bits, we have

$$|p^2| = 2048 \text{ bits.}$$

When the bit length of q approximates that of p , the modulus size enlarges:

$$|n| \approx 2048 + 1024 = 3072 \text{ bits.}$$

Table 6

Estimated storage of the encrypted distance matrix (424 bytes per entry).

Number of nodes	Number of entries	Estimated storage
$N = 1000$	10^6	0.36 GB
$N = 10,000$	10^8	35.8 GB
$N \approx 31,623$	10^9	358 GB
$N = 100,000$	10^{10}	3.49 TB

Since each ciphertext is an element of $\mathbb{Z}_n^*$, its bit-length is bounded by $\lceil \log_2 n \rceil + 1$,

which is approximately 3072 bits, 384 bytes.

Therefore, when the OU cryptosystem uses a 1024-bit public key (i.e., $|p| = 1024$ bits), the theoretical size of each ciphertext is approximately 384 bytes. Based on this value, we estimate the storage requirement of the encrypted matrix for different values of N . The results are shown in Table 6.

In a graph with 31,623 nodes (Table 6), approximating the scale of a medium-sized urban road network, the storage overhead of $\llbracket M \rrbracket$ is approximately 358 GB. Although considerably large, this overhead remains within the capacity of modern cloud infrastructures equipped with large-scale distributed storage systems.

6.2. Experimental analysis

We first compared the performances of PCSD and GESDQ at each stage of the shortest-distance query over the whole graph. We then modified GESDQ to handle PCSDQs and compared its performance with that of our scheme during the graph preprocessing, graph encryption, and query phases. Our scheme requires one graph-preprocessing execution, one encryption execution, and many queries. Therefore, the time requirement of the query dominantly affects the performance of the scheme.

The PRF was implemented in HMAC using the Python language on a laptop running the Windows 11 64-bit operating system, an i5-12400 2.5-GHz processor, and 16.0 GB of RAM.

Data Sets: We selected four different types of real datasets that are publicly available on the SNAP website¹: ego-Facebook, p2p-Gnutella05, wiki-Vote, and Twitch-PTBR. The specific graph dataset information is shown in Table 5, including the number of nodes, the number of edges, graph types (directed or undirected) and density of networks. Density represents the ratio of the number of edges to square of the number of nodes and reflects the tightness of the graph structure. The higher the density, the denser the edges between the nodes. As can be seen, the Twitch-PTBR network has the highest density, with the highest concentration of edges in the four datasets. Since PCSD and GESDQ work for both types of graphs, we applied the same test methods to all datasets.

These graphs contain tens of thousands of edges. Due to limited computing resources, we selected subgraphs with varying numbers of nodes and edges from the graphs test the performance of the schemes. The number of nodes and the number of edges contained in the selected subgraphs are shown in Fig. 4. We selected subgraphs containing 10–100 nodes and randomly assigned weights in the range of [1100]

¹ <http://snap.stanford.edu/data/>.

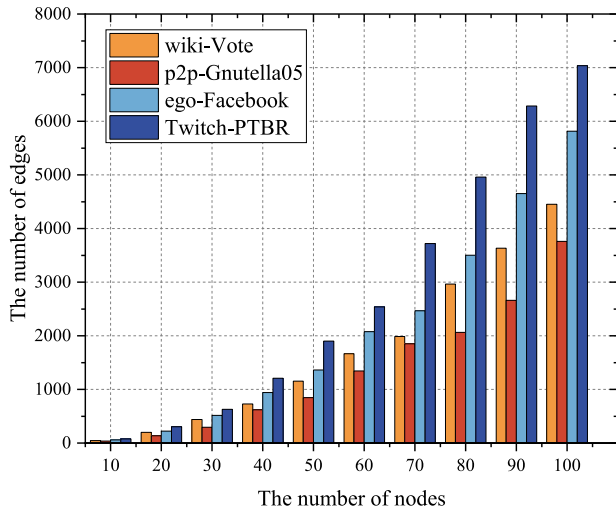


Fig. 4. The number of edges for each subgraph.

to each edge(weights represent distance). The execution time of the encryption and query phases for both schemes on these graphs was measured.

Comparison of shortest distance query over the whole graph: When there are no path nodes constraints, QU only uploads the start node and the end node can perform a privacy-preserving shortest distance query over the whole graph.

Graph processing: Both PCSD and GESDQ preprocess and encrypt the graph before the data is uploaded to the CS in order to perform operations on the encrypted graph while protecting the graph privacy. We performed PCSD and GESDQ on the four datasets. As shown in Fig. 5, the graph processing time for both PCSD and GESDQ increases with the number of nodes. The graph processing time of PCSD is higher than that of GESDQ. GESDQ transforms the graph into an adjacency table and encrypts it, leaving the time-consuming task to the query phase. In contrast, PCSD computes the shortest distance at preprocessing stage, which makes the processing time increase faster as nodes are increased. However, this makes query time shorter. In addition, graph processing is a one-time event, and its efficiency hardly affects the efficiency of the entire query.

To show in more detail the time of our scheme in the graph preprocessing phase, we tested the time of PCSD to process different numbers of nodes to generate the shortest distance matrix on four different datasets. As shown in Fig. 6, the shortest distance matrix generation time increases with the number of nodes. When the number of nodes is 100, the shortest distance matrix generation time is 1.4 s.

Querying: We compared PCSD and GESDQ for graph shortest distance query time on the whole graph. Due to the large difference in query times between PCSD and GESDQ, we use two graphs to show the query times of PCSD and GESDQ separately. Figs. 7 and 8 illustrate that query time increases as the number of graph nodes increase for PCSD and GESDQ, respectively.

In the shortest distance query on the whole graph, the query time of PCSD is in 14–17 ms range and is independent of the number of graph nodes. As shown in Fig. 7, the PCSD takes between 14–17 ms to perform a full-graph shortest distance query. On the contrary, the query time is affected by the subgraph structure as GESDQ needs to compute all the path distance between two nodes at the query stage. When $n = 100$, the query time of GESDQ is 280 s. To reduce the effect of graph structure, we perform multiple queries on each dataset to take the average value as the result. When $n = 100$, PCSD is 1.64×10^4 times more efficient than GESDQ. PCSD has better performance on large graphs.

Comparison of path-constrained shortest distance query: For path-constrained shortest distance queries, PCSD and GESDQ do the

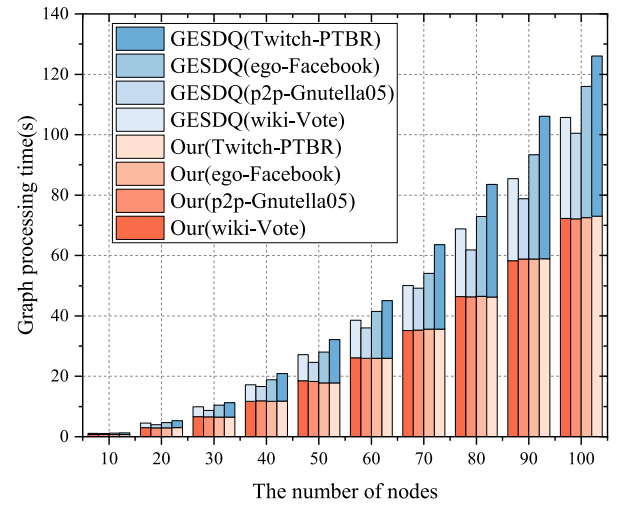


Fig. 5. The graph processing time of PCSD and GESDQ for different datasets.

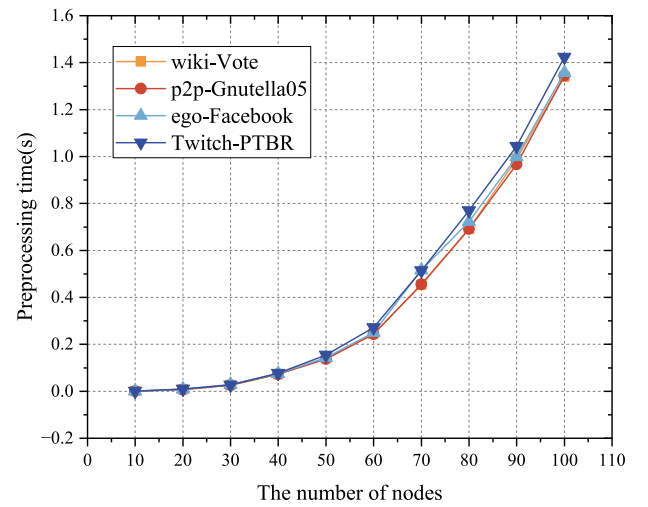


Fig. 6. The matrix generation time of PCSD for different datasets.

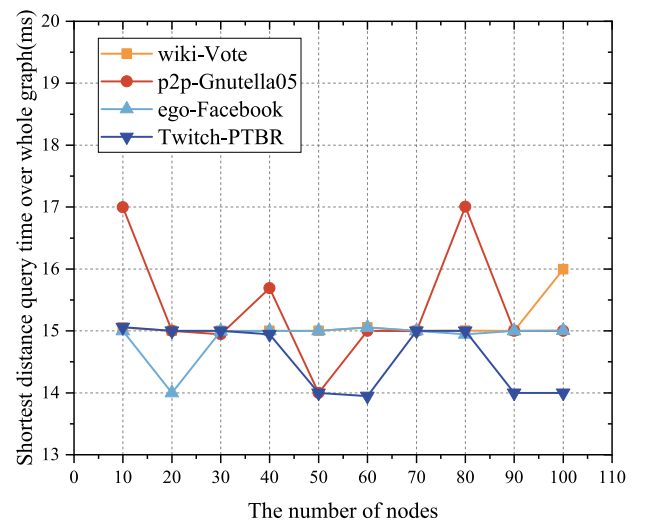


Fig. 7. The query time over the whole graph of PCSD for different datasets.

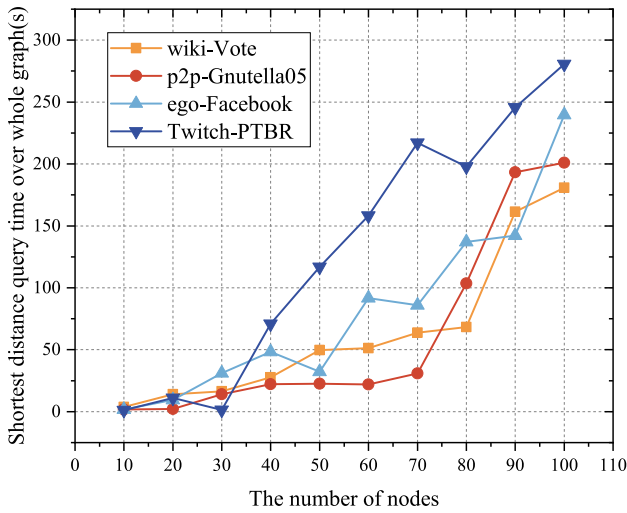


Fig. 8. The query time over the whole graph of GESDQ for different datasets.

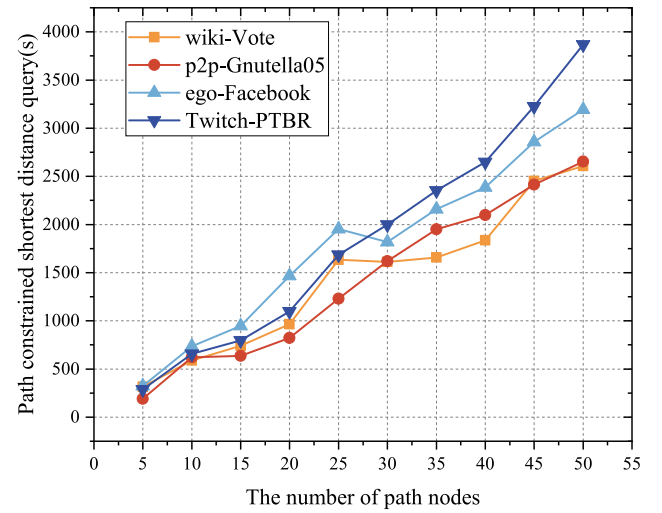


Fig. 10. The path constrained shortest distance query time of GESDQ for different datasets.

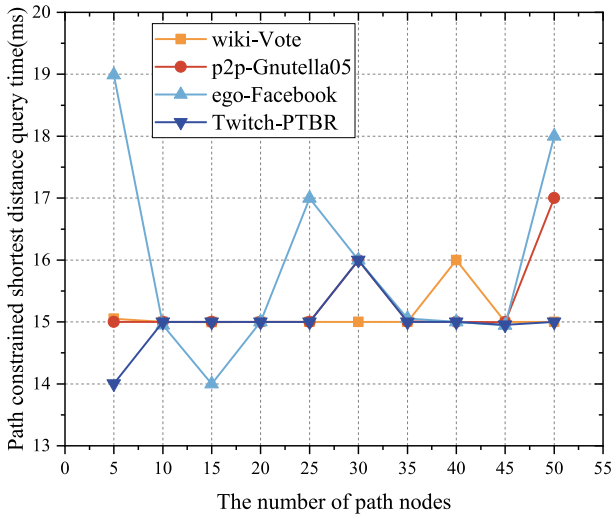


Fig. 9. The path constrained shortest distance query time of PCSD for different datasets.

same processing on the graph as for shortest path queries on the whole graph, and we do not show the results again. We executed PCSD and GESDQ queries on four different datasets and give the results in Figs. 9, 10 respectively. Figs. 9, 10 illustrate query time increase as number of path nodes increase for PCSD and GESDQ, respectively.

Querying: The querying performance was evaluated on randomly selected 5–50 query nodes from each of the four subgraph datasets (each graph containing 100 nodes). The results of PCSD and GESDQ are shown in Figs. 9 and 10, respectively. During the query phase, PCSD needs only to compare the pseudo-identity of the query nodes in the query token and the nodes in the encrypted graph. The final distance is then obtained through ciphertext multiplication. This process involves multiple modular multiplications but the decryption and re-encryption processes involve only three modular exponentiations, rendering them highly efficient. When finding the shortest distance with 50 path nodes, PCSD required only 20 ms. The GESDQ scheme, which continuously updates the shortest distances between nodes using a greedy strategy, required more iterations that lengthen the query time. When executing 50 shortest-distance queries between two nodes on the encrypted graph, GESDQ consumed 64.48 min (3868 s). That is, the query execution step is 1.9×10^5 faster in PCSD than in GESDQ.

7. Discussions and conclusions

7.1. Conclusions

We propose a privacy-preserving scheme for path-constrained shortest distance (PCSD) queries on encrypted graphs, leveraging homomorphic encryption and pseudo-random functions. The scheme improves query efficiency, supports multi-user authorized queries, and retains decryption rights with the Graph Owner.

7.2. Limitations

The main limitation is that the Graph Owner must stay online for decryption during each query, which may create bottlenecks for large-scale or concurrent queries. Sharing decryption keys with users could remove this requirement but would compromise security and multi-user support.

In practical deployment, however, the Graph Owner does not necessarily need to remain continuously online. To avoid inconvenience caused by constant availability requirements, a notification-based mechanism can be adopted. Similar to push notification systems in mobile applications, the cloud server can send a request notification to the Graph Owner only when decryption is required. The Graph Owner can then temporarily come online to process the request and go offline afterward. When no decryption demand exists, the Graph Owner does not need to stay online. This approach alleviates the operational burden while preserving the security model of our scheme.

7.3. Proxy re-encryption analysis

PRE could theoretically allow the cloud to re-encrypt data for authorized users without the Owner's active involvement. However, integrating PRE with our OU-based cryptosystem is non-trivial: OU does not natively support PRE, and adapting it would require major cryptographic modifications, increase system complexity, and introduce additional trust assumptions. Therefore, applying PRE remains an open research direction.

7.4. Future work

Future directions include designing PRE-compatible or alternative homomorphic schemes to remove the online requirement, improving dynamic authorization to reduce computational load, and extending the scheme for dynamic graphs and broader real-world applications.

CRediT authorship contribution statement

Yuwei Duan: Writing – original draft, Methodology, Formal analysis. **Jingjuan Yu:** Formal analysis. **Minghui Wang:** Data curation. **Shundong Li:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Reports a relationship with that includes: Has patent pending to. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Bondy, John Adrian, Murty, Uppaluri Siva Ramachandra, 2008. Graph Theory. Springer Publishing Company, Incorporated.
- Chase, Melissa, Kamara, Seny, 2010. Structured encryption and controlled disclosure. In: Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16. Springer, pp. 577–594.
- Cohen, Edith, Halperin, Eran, Kaplan, Haim, Zwick, Uri, 2003. Reachability and distance queries via 2-hop labels. SIAM J. Comput. 32 (5), 1338–1355.
- Gad-Elrab, Ahmed A.A., Noaman, Amin Y., 2020. A two-tier bipartite graph task allocation approach based on fuzzy clustering in cloud-fog environment. Future Gener. Comput. Syst. 103, 79–90.
- Ge, Xinrui, Yu, Jia, Zhang, Hanlin, Bai, Jianli, Fan, Jianxi, Xiong, Neal N, 2021. SPPS: A search pattern privacy system for approximate shortest distance query of encrypted graphs in IIoT. IEEE Trans. Syst. Man Cybern.: Syst. 52 (1), 136–150.
- Goldreich, Oded, 2004. Foundations of Cryptography, Volume 2. Cambridge university press Cambridge.
- Gong, Zengyang, Zeng, Yuxiang, Chen, Lei, 2024. Querying shortest path on large time-dependent road networks with shortcuts. In: 2024 IEEE 40th International Conference on Data Engineering. ICDE, IEEE, pp. 4532–4544.
- Guo, Jingjing, Sun, Jiacong, 2024. Secure shortest distance queries over encrypted graph in cloud computing. Wirel. Netw. 1–14.
- Hu, Chenfei, Zhang, Chuan, Lei, Dian, Wu, Tong, Liu, Ximeng, Zhu, Liehuang, 2023. Achieving privacy-preserving and verifiable support vector machine training in the cloud. IEEE Trans. Inf. Forensics Secur..
- Kumari, Saru, Li, Xiong, Wu, Fan, Das, Ashok Kumar, Choo, Kim-Kwang Raymond, Shen, Jian, 2017. Design of a provably secure biometrics-based multi-cloud-server authentication scheme. Future Gener. Comput. Syst. 68, 320–330.
- Li, Jing, 2020. Resource optimization scheduling and allocation for hierarchical distributed cloud service system in smart city. Future Gener. Comput. Syst. 107, 247–256.
- Liu, Chang, Zhu, Liehuang, He, Xiangjian, Chen, Jinjun, 2018. Enabling privacy-preserving shortest distance queries on encrypted graph data. IEEE Trans. Dependable Secur. Comput. 18 (1), 192–204.
- Luo, Yuchuan, Wang, Dongsheng, Fu, Shaojing, Xu, Ming, Chen, Yingwen, Huang, Kai, 2022. Approximate shortest distance queries with advanced graph analytics over large-scale encrypted graphs. In: 2022 18th International Conference on Mobility, Sensing and Networking. MSN, IEEE, pp. 287–294.
- Meng, Xianrui, Kamara, Seny, Nissim, Kobbi, Kollios, George, 2015. GreCs: Graph encryption for approximate shortest distance queries. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. pp. 504–517.
- Qiao, Miao, Cheng, Hong, Chang, Lijun, Yu, Jeffrey Xu, 2012. Approximate shortest distance computing: A query-dependent local landmark scheme. IEEE Trans. Knowl. Data Eng. 26 (1), 55–68.
- Selvaraj, D., Sankar, S.M., Dhinakaran, D., Anish, T.P., 2023. Outsourced analysis of encrypted graphs in the cloud with privacy protection. arXiv preprint arXiv: 2304.10833.
- Shen, Meng, Chen, Siqi, Zhu, Liehuang, Xiao, Renyi, Xu, Ke, Du, Xiaojiang, 2019. Privacy-preserving graph encryption for approximate constrained shortest distance queries. In: 2019 IEEE Global Communications Conference. GLOBECOM, IEEE, pp. 1–6.
- Shen, Meng, Ma, Baoli, Zhu, Liehuang, Mijumbi, Rashid, Du, Xiaojiang, Hu, Jiankun, 2017. Cloud-based approximate constrained shortest distance queries over encrypted graphs with privacy protection. IEEE Trans. Inf. Forensics Secur. 13 (4), 940–953.
- Sommer, Christian, 2014. Shortest-path queries in static networks. ACM Comput. Surv. 46 (4), 1–31.
- Sun, Fangyuan, Yu, Jia, Hao, Rong, Yang, Ming, Kong, Fanyu, 2023. Privacy-preserving shortest distance query for dynamic large-scale road network. IEEE Syst. J..
- Tatit, Pornrawee, Adhinugraha, Kiki, Taniar, David, 2024. Navigating the maps: Euclidean vs. Road network distances in spatial queries. Algorithms 17 (1), 29.
- Wang, Wenli, Jia, Zhuliang, Xu, Mengfan, Li, Shundong, 2022. SPCS: Strong privacy-preserving-constrained shortest distance queries on encrypted graphs. IEEE Internet Things J. 9 (22), 22516–22528.
- Wang, Qian, Ren, Kui, Du, Minxin, Li, Qi, Mohaisen, Aziz, 2017. SecGDB: Graph encryption for exact shortest distance queries with efficient updates. In: Financial Cryptography and Data Security: 21st International Conference, FC 2017, Sliema, Malta, April 3-7, 2017, Revised Selected Papers 21. Springer, pp. 79–97.
- Wei, Fang, 2010. TED: efficient shortest path query answering on graphs. In: Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data. pp. 99–110.
- Wei, Zongchao, 2022. High performance computing simulation of intelligent logistics management based on shortest path algorithm. Comput. Intell. Neurosci. 2022 (1), 7930553.
- Zhang, Can, Zhu, Liehuang, Xu, Chang, Sharif, Kashif, Zhang, Chuan, Liu, Ximeng, 2020. PGAS: Privacy-preserving graph encryption for accurate constrained shortest distance queries. Inform. Sci. 506, 325–345.
- Zhu, Andy Diwen, Ma, Hui, Xiao, Xiaokui, Luo, Siqiang, Tang, Youze, Zhou, Shuigeng, 2013. Shortest path and distance queries on road networks: towards bridging theory and practice. In: Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data. pp. 857–868.